

Guida al Midterm 1

Gaussian Mixture Model con EM Algorithm

Versione dettagliata con esempi concreti

GDL – Generative and Deep Learning (UniPI)

Indice

1	Panoramica del compito	2
1.1	Il dataset di training	2
2	Cos'è un GMM	3
2.1	Gaussiana multivariata con covarianza diagonale	3
3	Log-Likelihood: il cuore della valutazione	4
3.1	Le tre sommatorie: l, k, j	4
3.2	Sommatoria su l (feature): la log-Gaussiana	4
3.3	Sommatoria su k (componenti): la probabilità del campione	5
3.4	Sommatoria su j (campioni): la log-likelihood	6
3.5	Trucco Log-Sum-Exp: perché e come	6
4	La funzione <code>_compute_log_weighted</code>: dal concetto al codice	7
4.1	Dalla formula alla log-Gaussiana: i tre pezzi	7
4.2	PEZZO A: la costante $-\frac{d}{2} \log(2\pi)$	7
4.3	PEZZO B: $-\frac{1}{2} \sum_{l=1}^d \log \sigma_{k,l}^2$	7
4.4	PEZZO C: $-\frac{1}{2} \sum_{l=1}^d \frac{(x_{j,l} - \mu_{k,l})^2}{\sigma_{k,l}^2}$	8
4.5	Assemblare i tre pezzi	9
4.6	Da <code>log_weighted</code> a <code>log_likelihood</code>	9
5	Inizializzazione con K-Means++	9
5.1	L'algoritmo K-Means++ passo passo	10
5.2	Inizializzazione completa	10
6	Algoritmo EM: spiegazione dettagliata	11
6.1	E-Step (Expectation) – passo passo	11
6.2	M-Step (Maximization) – passo passo	12
6.3	Criterio di convergenza	13
6.4	Schema complessivo dell'algoritmo	14
7	BIC (Bayesian Information Criterion)	14
7.1	Conteggio dei parametri $ \theta $	15
8	Riepilogo operativo metodo per metodo	15
8.1	<code>__init__(self, n_categories, n_features)</code>	15
8.2	<code>fit(self, samples)</code>	16
8.3	<code>log_likelihood(self, samples) → scalare</code>	16
8.4	<code>bic(self, samples) → scalare</code>	16

9	Insidie comuni e consigli	17
10	Collegamento con le lezioni (GDL7 e GDL8)	17

1 Panoramica del compito

Si richiede di implementare un **Gaussian Mixture Model (GMM)** addestrato tramite l'algoritmo **Expectation-Maximization (EM)**, con selezione del modello tramite **BIC**. Nello specifico:

1. `log_likelihood(samples)` – calcola $\log P(\mathbf{X} | \boldsymbol{\theta})$
2. `fit(samples)` – addestra i parametri con EM
3. `bic(samples)` – calcola il Bayesian Information Criterion
4. Selezionare il miglior K tra $\{1, \dots, 6\}$ massimizzando il BIC

Vincolo importante

La matrice di covarianza è **diagonale**: per ogni Gaussiana k si memorizza solo un vettore di varianze $\boldsymbol{\sigma}_k$ di dimensione d (non una matrice $d \times d$).

1.1 Il dataset di training

Il file `train.csv` contiene:

- $N = 800$ campioni (righe)
- $d = 5$ feature (colonne): `x1`, `x2`, `x3`, `x4`, `x5`
- Valori continui a virgola mobile (es. -1.988 , 0.309 , ecc.)

Quando nel codice di training si fa `train_df = pd.read_csv('train.csv')`, si ottiene un DataFrame pandas. La riga `gmm.fit(train_df.values)` passa al metodo `fit` un **array numpy** di shape $(800, 5)$.

Come pensare ai dati

Immagina una tabella con 800 righe e 5 colonne. Ogni riga è un **punto** in uno spazio a 5 dimensioni. Il GMM cerca di spiegare questi 800 punti come generati da una miscela di K “nuvole” gaussiane, ognuna con il suo centro ($\boldsymbol{\mu}_k$), la sua dispersione ($\boldsymbol{\sigma}_k$) e il suo peso (π_k).

Notazione: come leggere gli indici

In tutto il documento:

- $j = 1, \dots, N$ indicizza i **campioni** (righe del dataset, $N = 800$)
- $k = 1, \dots, K$ indicizza le **componenti** Gaussiane (cluster)
- $l = 1, \dots, d$ indicizza le **feature**/dimensioni (colonne, $d = 5$)
- $\mathbf{x}_j \in \mathbb{R}^d$ è il j -esimo campione (un vettore di 5 numeri)
- $x_{j,l}$ è la l -esima feature del j -esimo campione (un singolo numero)

2 Cos'è un GMM

Un GMM modella i dati come provenienti da una **miscela di K distribuzioni Gaussianhe**. L'idea intuitiva: nei dati ci sono K “gruppi” (cluster), ognuno con una forma a campana multidimensionale. Non sappiamo a quale gruppo appartenga ciascun dato (variabile **latente/nascosta** z).

Il processo generativo per ogni dato $\mathbf{x}$ è:

1. **Scegli un cluster k** con probabilità π_k (es. 30% cluster 1, 50% cluster 2, 20% cluster 3)
2. **Genera $\mathbf{x}$** dalla Gaussiana $\mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ di quel cluster

La densità di probabilità per un singolo punto $\mathbf{x}$ è:

$$P(\mathbf{x} \mid \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k) \quad (1)$$

I **parametri del modello** $\boldsymbol{\theta} = (\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\sigma})$ sono:

- π_k : peso della k -esima componente (*mixing coefficient*), con $\sum_{k=1}^K \pi_k = 1$
_pi: array numpy di shape (K,), es. per $K = 3$: [0.3, 0.5, 0.2]
- $\boldsymbol{\mu}_k \in \mathbb{R}^d$: vettore media della k -esima Gaussiana (il “centro” del cluster)
_mu: array numpy di shape (K, d), es. per $K = 3, d = 5$: shape (3, 5)
- $\boldsymbol{\sigma}_k \in \mathbb{R}^d$: vettore di varianze diagonali (la “dispersione” del cluster lungo ogni asse)
_sigma: array numpy di shape (K, d), es. per $K = 3, d = 5$: shape (3, 5)

2.1 Gaussiana multivariata con covarianza diagonale

Poiché $\boldsymbol{\Sigma}_k$ è diagonale, la densità si **fattorizza** come prodotto sulle dimensioni. Questo è il punto chiave che semplifica enormemente il calcolo.

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k) = \prod_{l=1}^d \frac{1}{\sqrt{2\pi \sigma_{k,l}^2}} \exp\left(-\frac{(x_l - \mu_{k,l})^2}{2 \sigma_{k,l}^2}\right) \quad (2)$$

Esempio concreto con il nostro dataset ($d = 5$)

Supponiamo di avere la componente $k = 1$ con:

- $\boldsymbol{\mu}_1 = (-2.0, 0.5, -1.0, 2.0, -3.0)$ – il centro
- $\boldsymbol{\sigma}_1^2 = (1.5, 1.0, 2.0, 1.5, 1.0)$ – le varianze

Per il punto $\mathbf{x}_1 = (-1.99, -1.15, 0.31, 1.43, -4.13)$ dal dataset, calcoliamo:

$$\mathcal{N}(\mathbf{x}_1 \mid \boldsymbol{\mu}_1, \boldsymbol{\sigma}_1) = \underbrace{\frac{1}{\sqrt{2\pi \cdot 1.5}} e^{-\frac{(-1.99+2.0)^2}{2 \cdot 1.5}}}_{\text{dim. 1}} \times \underbrace{\frac{1}{\sqrt{2\pi \cdot 1.0}} e^{-\frac{(-1.15-0.5)^2}{2 \cdot 1.0}}}_{\text{dim. 2}} \times \dots$$

Ogni fattore è una Gaussiana **univariata** indipendente. Questo è il vantaggio della covarianza diagonale!

In **log-space** (fondamentale per l'implementazione):

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \sum_{l=1}^d \log \sigma_{k,l}^2 - \frac{1}{2} \sum_{l=1}^d \frac{(x_l - \mu_{k,l})^2}{\sigma_{k,l}^2} \quad (3)$$

Come tradurlo in numpy – ragionamento

L'equazione (3) con un singolo campione $\mathbf{x}_j$ e una singola componente k produce **uno scalare**. Ma in numpy puoi calcolarla per **tutti i campioni e tutte le componenti in una volta** usando il broadcasting:

- `samples` ha shape $(N, d) = (800, 5)$
- `_mu` ha shape (K, d)
- `_sigma` ha shape (K, d) (le varianze)

Se espandi `samples` a shape $(N, 1, d)$ e `_mu` a shape $(1, K, d)$, la differenza `samples[:, None, :] - _mu[None, :, :]` ha shape (N, K, d) e contiene tutte le differenze $(x_{j,l} - \mu_{k,l})$ per ogni j, k, l . Poi puoi sommare lungo l'asse d (l'ultimo) per ottenere una matrice (N, K) : la log-densità di ogni campione sotto ogni componente.

Che cosa salvare in `_sigma`?

Puoi scegliere se salvare le **varianze** $\sigma_{k,l}^2$ o le **deviazioni standard** $\sigma_{k,l}$. L'importante è essere coerenti. Consiglio: salva le **varianze** (σ^2), così le formule del M-step e della Gaussiana corrispondono direttamente. Quando calcoli la log-Gaussiana, il termine $\log \sigma_{k,l}^2$ diventa semplicemente `np.log(_sigma)`.

3 Log-Likelihood: il cuore della valutazione

La **log-likelihood** misura quanto bene il modello spiega i dati. Più è alta, meglio è.

3.1 Le tre sommatorie: l, k, j

Il calcolo della log-likelihood coinvolge **tre sommatorie annidate**, ognuna su un indice diverso. È fondamentale capire cosa fa ognuna:

Indice	Su cosa somma	Range	Cosa produce
l	le feature (colonne)	$1 \dots d = 5$	la log-Gaussiana di un singolo (j, k)
k	le componenti (cluster)	$1 \dots K$	la log-prob. di un singolo campione j
j	i campioni (righe)	$1 \dots N = 800$	la log-likelihood totale (scalare)

Vediamole una per una, dall'interno verso l'esterno.

3.2 Sommatoria su l (feature): la log-Gaussiana

Fissato un campione j e una componente k , la log-Gaussiana (eq. (3)) somma sulle $d = 5$ feature:

$$\log \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k) = \underbrace{-\frac{d}{2} \log(2\pi)}_{\text{costante}} - \underbrace{\frac{1}{2} \sum_{l=1}^d \log \sigma_{k,l}^2}_{\text{somma su le 5 feature}} - \underbrace{\frac{1}{2} \sum_{l=1}^d \frac{(x_{j,l} - \mu_{k,l})^2}{\sigma_{k,l}^2}}_{\text{somma su le 5 feature}}$$

Esempio numerico: $\log \mathcal{N}$ per $j = 1, k = 1, d = 5$

Con: $\mathbf{x}_1 = (-2.0, -1.2, 0.3, 1.4, -4.1)$, $\boldsymbol{\mu}_1 = (-2.0, 0.5, -1.0, 2.0, -3.0)$, $\boldsymbol{\sigma}_1^2 = (1.5, 1.0, 2.0, 1.5, 1.0)$

Termine 1 (costante): $-\frac{5}{2} \log(2\pi) = -2.5 \times 1.838 = -4.594$

Termine 2 (somma di 5 log-varianze, scritti uno per uno):

$$\begin{aligned} \sum_{l=1}^5 \log \sigma_{1,l}^2 &= \log(1.5) + \log(1.0) + \log(2.0) + \log(1.5) + \log(1.0) \\ &= 0.405 + 0 + 0.693 + 0.405 + 0 = 1.503 \end{aligned}$$

Moltiplicato per $-\frac{1}{2}$: -0.752

Termine 3 (somma di 5 scarti quadratici normalizzati, scritti uno per uno):

$$\begin{aligned} \sum_{l=1}^5 \frac{(x_{1,l} - \mu_{1,l})^2}{\sigma_{1,l}^2} &= \frac{(-2 + 2)^2}{1.5} + \frac{(-1.2 - 0.5)^2}{1.0} + \frac{(0.3 + 1)^2}{2.0} + \frac{(1.4 - 2)^2}{1.5} + \frac{(-4.1 + 3)^2}{1.0} \\ &= \frac{0}{1.5} + \frac{2.89}{1.0} + \frac{1.69}{2.0} + \frac{0.36}{1.5} + \frac{1.21}{1.0} \\ &= 0 + 2.89 + 0.845 + 0.24 + 1.21 = 5.185 \end{aligned}$$

Moltiplicato per $-\frac{1}{2}$: -2.593

Risultato: $\log \mathcal{N}(\mathbf{x}_1 | \boldsymbol{\mu}_1, \boldsymbol{\sigma}_1) = -4.594 + (-0.752) + (-2.593) = -7.939$

Questo singolo numero dice: “il campione $\mathbf{x}_1$ sotto la Gaussiana $k = 1$ ha log-densità -7.939 ”.

La sommatoria su l ha “consumato” le 5 dimensioni, producendo **un solo numero** per la coppia ($j = 1, k = 1$).

Calcolando questo per **ogni** combinazione di j e k , ottieni una matrice di shape (N, K) — la log-densità di ogni campione sotto ogni componente. La chiamiamo **log_gauss**.

3.3 Sommatoria su k (componenti): la probabilità del campione

Prima di sommare su k , aggiungi il peso $\log \pi_k$ a ogni colonna della matrice **log_gauss**:

$$\text{log_weighted}[j, k] = \log \pi_k + \text{log_gauss}[j, k]$$

Questa matrice ha ancora shape (N, K) e si chiama **log_weighted**. Ogni elemento contiene:

$$\text{log_weighted}[j, k] = \log \pi_k + \log \mathcal{N}(\mathbf{x}_j | \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k) = \log(\pi_k \cdot \mathcal{N}(\mathbf{x}_j | \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k))$$

Ora, per ottenere $\log P(\mathbf{x}_j | \boldsymbol{\theta})$, devi “ricombinare” le K componenti con il **logsumexp** sulla riga j :

$$\log P(\mathbf{x}_j | \boldsymbol{\theta}) = \log \left(\sum_{k=1}^K e^{\text{log_weighted}[j, k]} \right) = \text{logsumexp}_k(\text{log_weighted}[j, :])$$

Risultato: un vettore di N valori, uno per campione. Lo chiamiamo **log_denom**.

Esempio: dalla matrice al vettore, con $K = 3$

Supponiamo che per il campione $j = 1$ i tre valori di `log_weighted` siano:

$$\begin{array}{ccc} k = 1 & k = 2 & k = 3 \\ \hline -8.855 & -7.450 & -9.200 \end{array}$$

Applico `logsumexp`: $a^* = -7.450$

$$\begin{aligned} \log P(\mathbf{x}_1 | \boldsymbol{\theta}) &= -7.450 + \log(e^{-8.855+7.450} + e^0 + e^{-9.200+7.450}) \\ &= -7.450 + \log(e^{-1.405} + 1 + e^{-1.750}) = -7.450 + \log(0.245 + 1 + 0.174) \\ &= -7.450 + \log(1.419) = -7.450 + 0.350 = \mathbf{-7.100} \end{aligned}$$

La sommatoria su k ha “consumato” le K componenti, producendo **un solo numero** per il campione $j = 1$.

3.4 Sommatoria su j (campioni): la log-likelihood

Infine, sommi su tutti gli N campioni:

$$\log P(\mathbf{X} | \boldsymbol{\theta}) = \sum_{j=1}^N \text{log_denom}[j] \quad (4)$$

Risultato: **uno scalare**. Questo è il valore che restituisce `log_likelihood()`.

Riepilogo: le tre sommatorie riducono le dimensioni

Partenza:	dataset (N, d) + parametri (K, d)
$\xrightarrow{\sum_{l_j}}$	matrice <code>log_gauss</code> di shape (N, K) — consuma le d feature
$\xrightarrow{+\log \pi}$	matrice <code>log_weighted</code> di shape (N, K) — aggiunge i pesi
$\xrightarrow{\text{logsumexp}_k}$	vettore <code>log_denom</code> di shape $(N,)$ — consuma le K componenti
$\xrightarrow{\sum_j}$	scalare (log-likelihood) — consuma gli N campioni

3.5 Trucco Log-Sum-Exp: perché e come

Il problema. Nella sommatoria su k , dobbiamo calcolare $\log(\sum_k e^{a_k})$. Se a_k sono molto negativi (es. -500), allora $e^{a_k} \approx 0$ e il computer lo arrotonda a 0. Poi $\log(0) = -\infty$. Risultato: calcolo inutile.

La soluzione. Sottrarre il massimo prima di esponenziare:

$$\log\left(\sum_k e^{a_k}\right) = a^* + \log\left(\sum_k e^{a_k - a^*}\right), \quad a^* = \max_k a_k \quad (5)$$

Perché funziona. Dopo aver sottratto a^* , almeno un termine è $e^0 = 1$, e gli altri sono $e^{\text{negativo}} \leq 1$. La somma è quindi ≥ 1 , e il logaritmo non esplode.

Esempio numerico

Supponiamo $K = 3$ e per un certo campione j :

$$a_{j,1} = -502.3, \quad a_{j,2} = -498.1, \quad a_{j,3} = -501.7$$

Senza trucco: $e^{-502.3} + e^{-498.1} + e^{-501.7} \approx 0 + 0 + 0 = 0 \Rightarrow \log(0) = \text{errore!}$

Con trucco: $a^* = -498.1$

$$a_{j,1} - a^* = -4.2, \quad a_{j,2} - a^* = 0, \quad a_{j,3} - a^* = -3.6$$

$$\log(e^{-4.2} + e^0 + e^{-3.6}) = \log(0.015 + 1 + 0.027) = \log(1.042) = 0.041$$

Risultato finale: $-498.1 + 0.041 = -498.06$. Nessun problema numerico!

4 La funzione `_compute_log_weighted`: dal concetto al codice

Questa funzione calcola la matrice `log_weighted` di shape (N, K) descritta sopra. È il **mattone fondamentale** che viene riusato sia da `log_likelihood` sia dall'E-step di EM.

Ogni elemento della matrice contiene:

$$\text{log_weighted}[j, k] = \underbrace{\log \pi_k}_{\text{peso}} + \underbrace{\log \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k)}_{\text{log-Gaussiana (la somma su } l \text{)}}$$

4.1 Dalla formula alla log-Gaussiana: i tre pezzi

Ricordiamo la formula (3) per un singolo (j, k) :

$$\log \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k) = \underbrace{-\frac{d}{2} \log(2\pi)}_{\text{PEZZO A}} - \underbrace{\frac{1}{2} \sum_{l=1}^d \log \sigma_{k,l}^2}_{\text{PEZZO B}} - \underbrace{\frac{1}{2} \sum_{l=1}^d \frac{(x_{j,l} - \mu_{k,l})^2}{\sigma_{k,l}^2}}_{\text{PEZZO C}}$$

Vediamo come calcolare ciascuno in numpy, pezzo per pezzo.

4.2 PEZZO A: la costante $-\frac{d}{2} \log(2\pi)$

È un singolo numero, uguale per tutti i campioni e tutte le componenti. Con $d = 5$:

$$-\frac{5}{2} \log(2\pi) = -2.5 \times 1.838 = -4.594$$

In numpy: semplicemente `-0.5 * d * np.log(2 * np.pi)`.

4.3 PEZZO B: $-\frac{1}{2} \sum_{l=1}^d \log \sigma_{k,l}^2$

Questo pezzo dipende solo da k (non da j): le varianze sono proprietà della componente, non del campione.

`self._sigma` ha shape (K, d) , es. con $K = 3, d = 5$:

$$\text{_sigma} = \begin{pmatrix} \sigma_{1,1}^2 & \sigma_{1,2}^2 & \sigma_{1,3}^2 & \sigma_{1,4}^2 & \sigma_{1,5}^2 \\ \sigma_{2,1}^2 & \sigma_{2,2}^2 & \sigma_{2,3}^2 & \sigma_{2,4}^2 & \sigma_{2,5}^2 \\ \sigma_{3,1}^2 & \sigma_{3,2}^2 & \sigma_{3,3}^2 & \sigma_{3,4}^2 & \sigma_{3,5}^2 \end{pmatrix}$$

Operazioni:

1. `np.log(self._sigma)` — prende il log di ogni elemento. Shape: ancora (K, d)
2. `np.sum(..., axis=1)` — somma lungo l'asse 1, cioè **lungo le colonne** (le d feature). Questo fa la $\sum_{l=1}^d$. Shape risultante: $(K,)$ — un valore per componente

In numpy: `log_var_sum = np.sum(np.log(self._sigma), axis=1)` — shape $(K,)$.

Esempio con $K = 3, d = 5$

Se `_sigma = [[1.5, 1.0, 2.0, 1.5, 1.0], [0.8, 1.2, 0.5, 2.0, 1.5], [1.0, 1.0, 1.0, 1.0, 1.0]]`:
`np.log(_sigma)` → `[[0.41, 0, 0.69, 0.41, 0], [-0.22, 0.18, -0.69, 0.69, 0.41], [0, 0, 0, 0, 0]]`
`np.sum(..., axis=1)` → `[1.50, 0.37, 0]` — shape $(3,)$, un valore per ognuna delle 3 componenti.

4.4 PEZZO C: $-\frac{1}{2} \sum_{l=1}^d \frac{(x_{j,l} - \mu_{k,l})^2}{\sigma_{k,l}^2}$

Questo è il pezzo più complesso perché dipende sia da j sia da k . Scomponilo in sotto-operazioni:

Sotto-passo C1: le differenze $(x_{j,l} - \mu_{k,l})$

Hai `samples` di shape (N, d) e `_mu` di shape (K, d) . Vuoi calcolare la differenza per **ogni** combinazione (j, k, l) . Il trucco in numpy è aggiungere un asse per rendere le dimensioni compatibili:

- `samples[:, None, :]` — da shape (N, d) a shape $(N, 1, d)$. Il `None` aggiunge un asse “vuoto” in mezzo.
- `self._mu[None, :, :]` — da shape (K, d) a shape $(1, K, d)$. Il `None` aggiunge un asse all’inizio.
- La sottrazione produce shape (N, K, d) : numpy “replica” automaticamente lungo gli assi di dimensione 1.

Cosa contiene il risultato (N, K, d)

L'elemento $[j, k, l]$ contiene $x_{j,l} - \mu_{k,l}$, cioè “di quanto il campione j si discosta dal centro k lungo la feature l ”. Con $N = 800, K = 3, d = 5$ ottieni un array 3D di $800 \times 3 \times 5 = 12\,000$ numeri — tutte le differenze possibili!

Sotto-passo C2: eleva al quadrato

`diff_sq = diff ** 2` — shape ancora (N, K, d) . Ogni elemento ora contiene $(x_{j,l} - \mu_{k,l})^2$.

Sotto-passo C3: dividi per le varianze

`self._sigma` ha shape (K, d) . Per dividerlo con `diff_sq` di shape (N, K, d) , devi aggiungere un asse:

`self._sigma[None, :, :]` ha shape $(1, K, d) \Rightarrow$ numpy lo replica N volte.

`scaled = diff_sq / self._sigma[None, :, :]` — shape (N, K, d) . L'elemento $[j, k, l]$ ora contiene $\frac{(x_{j,l} - \mu_{k,l})^2}{\sigma_{k,l}^2}$.

Sotto-passo C4: somma sulle feature

`mahal = np.sum(scaled, axis=2)` — somma sull'asse 2 (l'ultimo, quello delle d feature). Questo fa la $\sum_{l=1}^d$. Shape risultante: (N, K) .

L'elemento $[j, k]$ contiene $\sum_{l=1}^d \frac{(x_{j,l} - \mu_{k,l})^2}{\sigma_{k,l}^2}$ — un singolo numero che dice “quanto è distante il campione j dalla Gaussiana k ”.

4.5 Assemblare i tre pezzi

$$\text{log_gauss} = \underbrace{-0.5 \cdot d \cdot \log(2\pi)}_{\text{PEZZO A: scalare}} + \underbrace{(-0.5) \cdot \text{log_var_sum}}_{\text{PEZZO B: shape (K,)}} + \underbrace{(-0.5) \cdot \text{mahal}}_{\text{PEZZO C: shape (N,K)}}$$

Il PEZZO B ha shape $(K,)$, ma deve sommarsi con il PEZZO C di shape (N, K) . Si usa `log_var_sum[None, :]` per portarlo a shape $(1, K)$, poi numpy lo replica per tutte le N righe. Risultato: `log_gauss` ha shape (N, K) .

Infine, aggiungi $\log \pi_k$:

`np.log(self._pi)` ha shape $(K,)$. Con `[None, :]` diventa $(1, K)$, poi broadcast a (N, K) .

```
log_weighted = log_gauss + np.log(self._pi)[None, :]
```

Shape finale: (N, K) .

4.6 Da log_weighted a log_likelihood

Ora hai la matrice `log_weighted` di shape (N, K) . Per la log-likelihood:

1. **Logsumexp per ogni riga** (sommatoria su k):

- Trova il massimo di ogni riga: `a_max = np.max(log_weighted, axis=1)` — shape $(N,)$
- Sottrai, esponenzia, somma, log:
`log_denom = a_max + np.log(np.sum(np.exp(log_weighted - a_max[:, None]), axis=1))`
Il `a_max[:, None]` trasforma shape $(N,)$ in $(N, 1)$ per poterlo sottrarre da ogni colonna della matrice (N, K) . Dopo `np.sum(..., axis=1)` si ottiene shape $(N,)$.

2. **Somma su tutti i campioni** (sommatoria su j):

```
return np.sum(log_denom) — uno scalare
```

La matrice `log_weighted` si riusa ovunque

La stessa matrice `log_weighted` e lo stesso vettore `log_denom` servono anche nell'E-step per calcolare le responsabilità (Sezione 6.1). Per questo conviene mettere il calcolo in una funzione separata `_compute_log_weighted`.

5 Inizializzazione con K-Means++

L'inizializzazione dei centri (μ) è cruciale. K-means++ sceglie K punti dal dataset in modo che siano **ben distanziati** tra loro, aumentando le probabilità che EM converga a una buona soluzione.

5.1 L'algoritmo K-Means++ passo passo

Algorithm 1 K-Means++ – inizializzazione dei centri

Require: Dataset $\mathbf{X} \in \mathbb{R}^{N \times d}$, numero di centri K

Ensure: Centri iniziali $\mu_1, \dots, \mu_K$

- 1: **Passo 1:** Scegli μ_1 uniformemente a caso tra gli N campioni
- 2: **for** $k = 2, \dots, K$ **do**
- 3: **Passo 2:** Per ogni campione $\mathbf{x}_j$, calcola la distanza al centro più vicino **già scelto**:

$$D(\mathbf{x}_j) = \min_{k' < k} \|\mathbf{x}_j - \mu_{k'}\|^2$$

- 4: **Passo 3:** Scegli μ_k tra i campioni con probabilità proporzionale a $D(\mathbf{x}_j)$:

$$P(\text{scegli } \mathbf{x}_j) = \frac{D(\mathbf{x}_j)}{\sum_{i=1}^N D(\mathbf{x}_i)}$$

- 5: **end for**
-

Esempio con $K = 3$ e i nostri dati

1. **Passo 1:** Scegli a caso il campione $\mathbf{x}_{42}$ come μ_1
2. **Per** $k = 2$: Calcola la distanza di ogni campione da μ_1 . I campioni più lontani hanno più probabilità di essere scelti. Supponiamo venga scelto $\mathbf{x}_{317}$ come μ_2
3. **Per** $k = 3$: Calcola per ogni campione la distanza **minima** da μ_1 e μ_2 . I campioni lontani da *entrambi* i centri già scelti hanno più probabilità. Viene scelto $\mathbf{x}_{150}$ come μ_3

Risultato: tre centri ben separati nello spazio 5-dimensionale!

Ragionamento per l'implementazione

La distanza (euclidea al quadrato) tra un campione $\mathbf{x}_j$ e un centro $\mu_{k'}$ con $d = 5$ feature è:

$$\|\mathbf{x}_j - \mu_{k'}\|^2 = \sum_{l=1}^5 (x_{j,l} - \mu_{k',l})^2$$

Per la scelta probabilistica: una volta calcolate tutte le distanze $D(\mathbf{x}_j)$, le normalizzi dividendo per la loro somma (ottiene una distribuzione di probabilità), e poi campioni un indice secondo quella distribuzione (`np.random.choice` con il parametro `p`).

5.2 Inizializzazione completa

Dopo aver ottenuto i centri con K-means++, si inizializzano tutti i parametri:

1. π : pesi uniformi $\pi_k = 1/K$ per ogni k
2. μ : i K centri ottenuti da K-means++
3. σ : diverse opzioni ragionevoli:
 - **Varianza globale:** calcola la varianza di tutti i dati lungo ogni dimensione, e usala per tutte le K componenti. Formalmente: $\sigma_{k,l}^2 = \text{Var}(\mathbf{X}_{:,l})$ per ogni k .

- **Varianze unitarie:** $\sigma_{k,l}^2 = 1$ per tutti.

Esempio concreto per $K = 3, d = 5$

Dopo l'inizializzazione:

- `_pi` = `[1/3, 1/3, 1/3]` – shape (3,)
- `_mu` = `[[centro1], [centro2], [centro3]]` – shape (3, 5), tre punti dal dataset
- `_sigma` = `[[var_x1, var_x2, ..., var_x5], [var_x1, ...], [var_x1, ...]]`
– shape (3, 5), stesse varianze per ogni componente

6 Algoritmo EM: spiegazione dettagliata

L'idea di fondo dell'EM: se sapessimo a quale cluster appartiene ogni punto, potremmo calcolare i parametri con semplici medie pesate (come nel caso “fully observed” della lezione GDL7). Ma non lo sappiamo! Allora:

1. **E-step:** “fingi” di saperlo, stimando le probabilità di appartenenza (responsabilità)
2. **M-step:** usa quelle stime per aggiornare i parametri come se i dati fossero completi
3. **Ripeti:** le stime migliorano $\rightarrow$ i parametri migliorano $\rightarrow$ le stime migliorano $\rightarrow \dots$

6.1 E-Step (Expectation) – passo passo

Obiettivo: calcolare la matrice delle **responsabilità** γ di shape (N, K) .

L'elemento $\gamma_{j,k}$ risponde alla domanda: “data l'osservazione $\mathbf{x}_j$ e i parametri attuali, qual è la probabilità che $\mathbf{x}_j$ provenga dalla componente k ?”

$$\gamma_{j,k} = P(z_j = k \mid \mathbf{x}_j, \boldsymbol{\theta}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k)}{\sum_{k'=1}^K \pi_{k'} \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_{k'}, \boldsymbol{\sigma}_{k'})} \quad (6)$$

Procedura concreta:

1. **Calcola la matrice** `log_weighted` di shape (N, K) :

$$\text{log_weighted}[j, k] = \log \pi_k + \log \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k)$$

Questo è esattamente la stessa matrice di cui parlavo nella Sezione 3. La log-Gaussiana si calcola con l'eq. (3).

2. **Calcola il denominatore** (un vettore di N valori) applicando `logsumexp` lungo l'asse delle K componenti:

$$\text{log_denom}[j] = \text{logsumexp}_k(\text{log_weighted}[j, :])$$

3. **Calcola γ** sottraendo ed esponenziando:

$$\gamma_{j,k} = \exp(\text{log_weighted}[j, k] - \text{log_denom}[j])$$

Esempio numerico per un campione j con $K = 3$

Supponiamo:

$$\text{log_weighted}[j, :] = [-502.3, -498.1, -501.7]$$

1. $\text{log_denom}[j] = \text{logsumexp}(-502.3, -498.1, -501.7) = -498.06$ (vedi esempio Sez. 3.1)
2. $\gamma_{j,1} = e^{-502.3 - (-498.06)} = e^{-4.24} = 0.014$
3. $\gamma_{j,2} = e^{-498.1 - (-498.06)} = e^{-0.04} = 0.960$
4. $\gamma_{j,3} = e^{-501.7 - (-498.06)} = e^{-3.64} = 0.026$

Verifica: $0.014 + 0.960 + 0.026 = 1.0$ ✓

Interpretazione: il campione $\mathbf{x}_j$ appartiene quasi sicuramente alla componente 2 (96%).

Controllo di correttezza

Ogni riga di γ deve sommare a 1 (a meno di errori di arrotondamento). Se non somma a 1, c'è un bug.

6.2 M-Step (Maximization) – passo passo

Obiettivo: aggiornare π, μ, σ usando le responsabilità γ appena calcolate.

Passo 0 – Conta effettiva: Per ogni componente k , calcola quanti campioni “le appartengono” (in senso pesato):

$$N_k = \sum_{j=1}^N \gamma_{j,k} \quad (7)$$

Questo è un vettore di K valori. Rappresenta il “numero effettivo” di punti assegnati a ciascun cluster. La somma di tutti gli N_k dà N .

Esempio con $K = 3, N = 800$

Dopo l'E-step, potremmo ottenere $N_1 = 240.5, N_2 = 380.2, N_3 = 179.3$.

Nota che sono numeri reali (non interi!) perché i punti hanno appartenenza “morbida” a più cluster. $240.5 + 380.2 + 179.3 = 800$ ✓

Passo 1 – Aggiorna i pesi:

$$\pi_k^{(\text{new})} = \frac{N_k}{N} \quad (8)$$

La frazione di campioni “assegnati” a ogni componente. Per l'esempio sopra: $\pi_1 = 0.301, \pi_2 = 0.475, \pi_3 = 0.224$.

Passo 2 – Aggiorna le medie:

$$\mu_{k,l}^{(\text{new})} = \frac{\sum_{j=1}^N \gamma_{j,k} x_{j,l}}{N_k} \quad (9)$$

È una **media pesata** dei dati, dove i pesi sono le responsabilità. I campioni che “appartengono di più” alla componente k (alti $\gamma_{j,k}$) pesano di più nel determinare il centro.

Ragionamento per l'implementazione

L'eq. (9) per tutte le componenti e tutte le dimensioni si può scrivere come un prodotto matrice:

- γ^T ha shape (K, N)
- $\mathbf{X}$ ha shape (N, d)
- $\gamma^T \cdot \mathbf{X}$ ha shape (K, d) – esattamente la forma di $\boldsymbol{\mu}$!

Poi dividi ogni riga k per N_k .

Passo 3 – Aggiorna le varianze:

$$\sigma_{k,l}^{2(\text{new})} = \frac{\sum_{j=1}^N \gamma_{j,k} (x_{j,l} - \mu_{k,l}^{(\text{new})})^2}{N_k} \quad (10)$$

È la **varianza pesata** dei dati rispetto al nuovo centro. Nota: si usano le medie $\boldsymbol{\mu}^{(\text{new})}$ appena calcolate!

Ragionamento per l'implementazione

Il termine $(x_{j,l} - \mu_{k,l}^{(\text{new})})^2$ per tutti i j, k, l si ottiene con lo stesso trucco del broadcasting:

- `diff = samples[:, None, :] - mu_new[None, :, :]` ha shape (N, K, d)
- `diff_sq = diff ** 2` ha shape (N, K, d)
- Moltiplica per $\gamma_{j,k}$ (espanso a shape $(N, K, 1)$), somma sull'asse j (asse 0), e dividi per N_k

Floor sulle varianze – FONDAMENTALE

Dopo aver aggiornato σ , applica un valore minimo:

$$\sigma_{k,l}^2 \leftarrow \max(\sigma_{k,l}^2, \epsilon), \quad \epsilon = 10^{-6}$$

Senza questo, una varianza a zero causa $\log(0) = -\infty$ e divisione per zero nella log-Gaussiana.

6.3 Criterio di convergenza

Dopo ogni iterazione di E+M, calcola la log-likelihood (eq. (4)). Fermati quando:

$$|\log \mathcal{L}^{(t+1)} - \log \mathcal{L}^{(t)}| < \varepsilon \quad (\text{es. } \varepsilon = 10^{-6}) \quad (11)$$

oppure si raggiunge un numero massimo di iterazioni (es. 200).

Teorema di Dempster, Laird e Rubin (1977)

L'algoritmo EM garantisce che la log-likelihood sia **monotonicamente non-decrescente** ad ogni iterazione:

$$\log \mathcal{L}(\boldsymbol{\theta}^{(t+1)} \mid \mathbf{X}) \geq \log \mathcal{L}(\boldsymbol{\theta}^{(t)} \mid \mathbf{X})$$

Se la log-likelihood scende, c'è un bug. Attenzione: “non-decrescente” non significa che raggiunga il massimo globale. EM può restare intrappolato in un massimo **locale**.

6.4 Schema complessivo dell'algoritmo

Algorithm 2 EM per GMM con covarianza diagonale – versione dettagliata

Require: $\mathbf{X} \in \mathbb{R}^{N \times d}$ (il dataset), K (numero componenti), ε (tolleranza), $T_{\max}$ (max iter.)

Ensure: Parametri ottimizzati $\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\sigma}$

```
1: Inizializza  $\boldsymbol{\mu}$  con K-means++,  $\boldsymbol{\pi} \leftarrow [1/K, \dots, 1/K]$ ,  $\boldsymbol{\sigma} \leftarrow$  varianza globale
2: prev_ll  $\leftarrow -\infty$ 
3: for  $t = 1, \dots, T_{\max}$  do
    — E-Step —
4:   for ogni campione  $j$  e componente  $k$  do
5:      $\log\_w[j, k] \leftarrow \log \pi_k + \log \mathcal{N}(\mathbf{x}_j \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k)$  (eq. (3))
6:   end for
7:   for ogni campione  $j$  do
8:      $\log\_d[j] \leftarrow \text{logsumexp}_k(\log\_w[j, :])$  (eq. (5))
9:   end for
10:   $\gamma_{j,k} \leftarrow \exp(\log\_w[j, k] - \log\_d[j])$  per ogni  $j, k$ 
    — M-Step —
11:   $N_k \leftarrow \sum_j \gamma_{j,k}$  per ogni  $k$ 
12:   $\pi_k \leftarrow N_k / N$  per ogni  $k$ 
13:   $\mu_{k,l} \leftarrow \sum_j \gamma_{j,k} x_{j,l} / N_k$  per ogni  $k, l$  (eq. (9))
14:   $\sigma_{k,l}^2 \leftarrow \sum_j \gamma_{j,k} (x_{j,l} - \mu_{k,l})^2 / N_k$  per ogni  $k, l$  (eq. (10))
15:   $\sigma_{k,l}^2 \leftarrow \max(\sigma_{k,l}^2, 10^{-6})$  (floor)
    — Convergenza —
16:  ll  $\leftarrow \sum_j \log\_d[j]$  (la log-likelihood è la somma dei log-denominatori!)
17:  if  $|ll - \text{prev\_ll}| < \varepsilon$  then
18:    break
19:  end if
20:  prev_ll  $\leftarrow ll$ 
21: end for
```

Osservazione cruciale – riga 15

Il vettore $\log_d[j]$ calcolato nell'E-step è esattamente $\log P(\mathbf{x}_j \mid \boldsymbol{\theta})$! Quindi la log-likelihood è semplicemente la somma di quei valori. Non serve ricalcolarla separatamente: è un sottoprodotto gratuito dell'E-step.

7 BIC (Bayesian Information Criterion)

Il BIC bilancia la bontà di adattamento con la complessità del modello:

$$\boxed{\text{BIC} = \log P(\mathbf{X} \mid \boldsymbol{\theta}) - \frac{|\boldsymbol{\theta}|}{2} \log n} \quad (12)$$

dove:

- $\log P(\mathbf{X} \mid \boldsymbol{\theta})$ è la log-likelihood (calcolata sopra)
- $n = N = 800$ è il numero di campioni
- $|\boldsymbol{\theta}|$ è il **numero totale di parametri liberi**

7.1 Conteggio dei parametri $|\boldsymbol{\theta}|$

Con K componenti, dati d -dimensionali e covarianza diagonale:

Parametri	Formula	Es. $K=3, d=5$	Spiegazione
π (pesi)	$K - 1$	2	vincolo $\sum = 1$, solo $K-1$ liberi
μ (medie)	$K \cdot d$	15	K vettori di dimensione d
σ (var. diag.)	$K \cdot d$	15	K vettori di d varianze
Totale	$(K-1) + 2Kd$	32	

Esempio: BIC per diversi K con $N = 800, d = 5$

K	$ \boldsymbol{\theta} $	Penalità $\frac{ \boldsymbol{\theta} }{2} \log 800$	LL (ipotetica)	BIC
1	$0 + 10 = 10$	33.4	-5200	-5233.4
2	$1 + 20 = 21$	70.2	-4800	-4870.2
3	$2 + 30 = 32$	106.9	-4500	-4606.9
4	$3 + 40 = 43$	143.7	-4480	-4623.7
5	$4 + 50 = 54$	180.4	-4475	-4655.4
6	$5 + 60 = 65$	217.2	-4472	-4689.2

In questo esempio il BIC migliore (più alto, meno negativo) è $K = 3$: la LL migliora molto da 1 a 3, ma da 3 in poi il guadagno non compensa la penalità di complessità.

Selezione del modello

Si vuole **massimizzare** il BIC (cioè trovare il valore **meno negativo**). Il modello con il BIC più alto è quello migliore. Il termine $-\frac{|\boldsymbol{\theta}|}{2} \log n$ penalizza modelli con troppi parametri (K alto), evitando l'overfitting.

8 Riepilogo operativo metodo per metodo

8.1 `__init__(self, n_categories, n_features)`

Questa funzione viene chiamata **prima** di `fit`. Puoi solo preparare le strutture dati qui, oppure rimandare l'inizializzazione vera (che dipende dai dati) all'inizio di `fit`.

1. Salva `n_categories` ($= K$) e `n_features` ($= d$)
2. Puoi mettere `_pi`, `_mu`, `_sigma` a `None` qui e inizializzarli in `fit` quando hai accesso ai dati

8.2 `fit(self, samples)`

Input: `samples` è un array numpy di shape $(N, d) = (800, 5)$.

1. **Inizializza** μ con K-means++ (usando i `samples`)
2. **Inizializza** π a $[1/K, \dots, 1/K]$
3. **Inizializza** σ alla varianza globale di ogni feature
4. **Loop EM**: ripeti per max iterazioni o fino a convergenza:
 - i. Calcola `log_gauss` di shape (N, K) con eq. (3)
 - ii. Calcola `log_weighted` = $\log \pi + \log_{\text{gauss}}$, shape (N, K)
 - iii. Calcola `log_denom` applicando `logsumexp` lungo asse 1, shape $(N,)$
 - iv. Calcola $\gamma = \exp(\log_{\text{weighted}} - \log_{\text{denom}}[:, \text{None}])$, shape (N, K)
 - v. Calcola $N_k = \gamma.\text{sum}(\text{axis}=0)$, shape $(K,)$
 - vi. Aggiorna $\pi = N_k / N$
 - vii. Aggiorna $\mu = \gamma^T \cdot \mathbf{X} / N_k[:, \text{None}]$
 - viii. Aggiorna σ con eq. (10), applica floor
 - ix. Calcola `ll` = `log_denom.sum()` (la log-likelihood)
 - x. Se $|\text{ll} - \text{prev_ll}| < \varepsilon$: break

8.3 `log_likelihood(self, samples) → scalare`

Input: array numpy di shape (N, d) . Può essere il train set o il test set.

1. Calcola `log_gauss` di shape (N, K) usando i parametri già fittati
2. Calcola `log_weighted` = $\log \pi + \log_{\text{gauss}}$
3. Applica `logsumexp` lungo l'asse delle componenti $\Rightarrow$ vettore di shape $(N,)$
4. Restituisci la **somma** $\Rightarrow$ uno scalare

Riuso del codice

La logica di `log_likelihood` è identica ai passi (i)–(iii) dell'E-step, seguiti dalla somma. Conviene scrivere una funzione ausiliaria che calcola la matrice `log_weighted` e il vettore `log_denom`, e richiamarla sia da `fit` che da `log_likelihood`.

8.4 `bic(self, samples) → scalare`

1. `ll` $\leftarrow$ `self.log_likelihood(samples)`
2. $|\theta| \leftarrow (\text{n_categories} - 1) + 2 \cdot \text{n_categories} \cdot d$
3. $n \leftarrow$ numero di righe di `samples`
4. Restituisci `ll` $- \frac{|\theta|}{2} \cdot \log(n)$ (usa log naturale, cioè `np.log`)

9 Insidie comuni e consigli

Errori frequenti

1. **Dimenticare il log-sum-exp**: se calcoli la Gaussiana senza log e poi fai il prodotto, i numeri vanno a zero (underflow). Lavora SEMPRE in log-space.
2. **Varianze a zero**: una componente che “cattura” pochissimi punti può avere varianza $\rightarrow 0$. Applica sempre il floor $\sigma_{k,l}^2 \geq 10^{-6}$.
3. **Confondere σ^2 e σ** : se in `_sigma` salvi le deviazioni standard ma nelle formule usi le varianze (o viceversa), i risultati sono sbagliati. Scegli una convenzione e mantienila.
4. **Broadcasting errato**: il broadcasting di numpy è potente ma insidioso. Controlla sempre le shape intermedie con `.shape`.
5. **Non aggiornare μ prima di σ** : la formula (10) usa le medie **nuove**. Se usi quelle vecchie, la log-likelihood potrebbe non crescere monotonamente.
6. **Log-likelihood che non cresce**: se la LL scende tra un'iterazione e l'altra, c'è un bug (vedi teorema EM). Stampa la LL ad ogni iterazione durante il debug.
7. **Inizializzazione nell'__init__**: non puoi fare K-means++ nell'__init__ perché non hai ancora i dati. Fallo all'inizio di `fit`.

10 Collegamento con le lezioni (GDL7 e GDL8)

GDL7 – Learning with fully observed variables: Introduce il Maximum Likelihood Estimation nel caso in cui tutte le variabili sono osservate. Con dati completi, i parametri ML si ottengono in forma chiusa (es. media campionaria, varianza campionaria). Il midterm estende questo al caso con variabili latenti.

GDL8 – Learning with hidden variables: Affronta il caso in cui esistono **variabili latenti** (come z nel GMM). La log-likelihood non si può massimizzare direttamente, quindi si usa l'algoritmo EM:

- Si introduce la *complete likelihood* $\mathcal{L}_c(\boldsymbol{\theta} \mid \mathbf{X}, \mathbf{Z})$
- L'E-step calcola l'**aspettativa** della complete log-likelihood rispetto alla distribuzione posteriore delle variabili latenti:

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \sum_{j=1}^N \sum_{k=1}^K P(z_j = k \mid \mathbf{x}_j, \boldsymbol{\theta}^{(t)}) \log(\pi_k P(\mathbf{x}_j \mid \mu_k, \sigma_k))$$

I termini $P(z_j = k \mid \mathbf{x}_j, \boldsymbol{\theta}^{(t)})$ sono esattamente le responsabilità $\gamma_{j,k}$.

- L'M-step **massimizza** Q rispetto a $\boldsymbol{\theta}$, derivando e ponendo a zero:

$$\boldsymbol{\theta}^{(t+1)} = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$$

Da cui si ottengono le formule chiuse (9) e (10).

- Il teorema di Dempster–Laird–Rubin garantisce convergenza monotona della log-likelihood.